

DR. B.R. Ambedkar National Institute of Technology Jalandhar



ARTIFICIAL INTELLIGENCE LAB (IT-515)

Submitted to:

Dr. Simranjit Singh

Assistant professor

Submitted by:

Granth Gupta

M.Tech-D.A

26201812

LAB-1 / QUESTION 1

Aim

To develop a simple rule-based expert system that uses facts and IF–THEN rules to infer the type of animal from the given characteristics.

CODE

```
# RULES
```

```
rules = [  
    ("has_hair", "mammal"),  
    ("has_feathers", "bird"),  
    ("gives_milk", "mammal"),  
    ("lays_eggs", "has_feathers", "bird")  
]
```

```
# Check IF all conditions in a rule match the given facts
```

```
def checkTarget(target, rules):
```

```
    for conditions, fact in rules:
```

```
        if target == fact:
```

```
            return conditions
```

```
    return []
```

```
# ANIMAL IDENTIFICATION FUNCTION
```

```
def identifyAnimal(given_facts, rules) -> str:
```

```
    for conditions, animal_type in rules:
```

```
        match = True
```

```
        for attribute in conditions:
```

```
            if attribute not in given_facts:
```

```
                match = False
```

else:

return "Current properties are insufficient to identify the animal"

if match:

return animal_type

return "Current properties are insufficient to identify the animal"

TESTING WITH USER INPUT

user_input = input("Enter facts separated by spaces (e.g., has_hair eats_meat):")

given_facts = user_input.strip().split()

result = identifyAnimal(given_facts, rules)

print("Type of animal: ", result)

OUTPUT

```
Granth Gupta@DESKTOP-OH4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-1/Improved_Code/1.py
Enter facts separated by spaces (e.g., has_hair eats_meat): has_hair gives_milk
Type of animal: mammal
```

LAB-1 / QUESTION 2

Aim

Develop an interactive expert system for student course recommendation. The system should ask the user questions about marks, interests, and skills and recommend a suitable course using predefined IF–THEN rules.

CODE

```
# RULES
```

```
RULES = [  
    ({"interest": "technology", "skill": "logic", "min_marks": 75}, "BSc in CSE"),  
    ({"interest": "healthcare", "skill": "research", "min_marks": 70}, "BSc in  
    Biotechnology"),  
    ({"interest": "business", "skill": "logic", "min_marks": 60}, "BBA in  
    Finance"),  
    ({"interest": "business", "skill": "communication", "min_marks": 60}, "BBA in  
    Marketing")  
]
```

```
# RECOMMENDATION FUNCTION
```

```
def recommendCourse(facts: dict) -> str:
```

```
    for conditions, course in RULES:
```

```
        interest_match = facts["interest"] == conditions["interest"]
```

```
        skill_match = facts["skill"] == conditions["skill"]
```

```
        marks_match = facts["marks"] >= conditions["min_marks"]
```

```
        if interest_match and skill_match and marks_match:
```

```
            return "Recommended course is " + course
```

```
    else:
```

```
return "Sorry! I don't have a recommended course for you."
```

```
# TESTING WITH USER INPUT
```

```
marks = float(input("Enter your marks: "))
```

```
interest = input("Enter your interest (technology, healthcare, business): ")
```

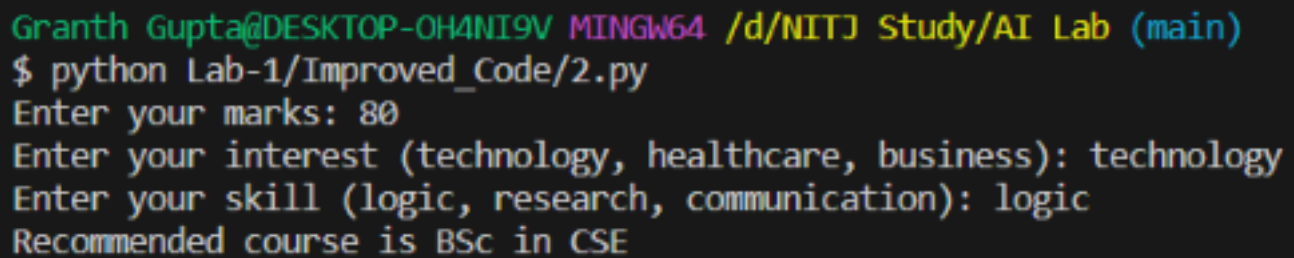
```
skill = input("Enter your skill (logic, research, communication): ")
```

```
user_facts = {  
    "marks": marks,  
    "interest": interest,  
    "skill": skill  
}
```

```
result = recommendCourse(user_facts)
```

```
print(result)
```

OUTPUT



```
Granth Gupta@DESKTOP-OH4NI9V MINGW64 /d/NITJ Study/AI Lab (main)  
$ python Lab-1/Improved_Code/2.py  
Enter your marks: 80  
Enter your interest (technology, healthcare, business): technology  
Enter your skill (logic, research, communication): logic  
Recommended course is BSc in CSE
```

LAB-1 / QUESTION 3

Aim

Implement a forward-chaining inference engine for an expert system. Given a set of initial facts and IF–THEN rules, repeatedly apply applicable rules to derive new facts until the required conclusion is reached or no further rules can be applied.

CODE

```
# KNOWN FACTS
```

```
known_facts_1 = ["has_fever", "has_cough"]
```

```
known_facts_2 = ["has_cough"]
```

```
# RULES
```

```
rules = [  
    (["has_fever", "has_cough"], "has_flu"),  
    (["has_flu"], "needs_rest")  
]
```

```
def checkFacts(conditions, facts):
```

```
    for cond in conditions:
```

```
        if cond not in facts:
```

```
            return False
```

```
    return True
```

```
# FORWARD CHAIN
```

```
def forward_chaining(facts, rules, goal):
```

```
    facts_changed = True
```

```
    while facts_changed:
```

```
        facts_changed = False
```

for conditions, result in rules:

if checkFacts(conditions, facts) and result not in facts:

facts.append(result)

facts_changed = True

if result == goal:

print(f'Goal '{goal}' successfully reached!')

return True

if goal in facts:

return True

else:

print(f'Could not reach the goal '{goal}'.')

return False

TESTING

target1 = "needs_rest"

print("Test 1")

print("Initial Facts: ", known_facts_1)

print("Target: ", target1)

forward_chaining(known_facts_1, rules, target1)

print("-----")

OUTPUT

```
Granth Gupta@DESKTOP-OH4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-1/3_.py
Test 1
Initial Facts: ['has_fever', 'has_cough']
Taregt: needs_rest
Goal 'needs_rest' successfully reached!
```

LAB-1 / QUESTION 4

Aim

Develop an expert system for fault diagnosis using backward chaining. Given a suspected fault as the goal, identify the rules and facts required to prove or reject the hypothesis. The system should also display an explanation showing how the conclusion was reached.

CODE

```
def checkTarget(target, rules):
    for conditions, fact in rules:
        if target == fact:
            return conditions
    return []

def backward_chaining(facts, rules, goal):
    if goal in facts:
        print(f"Fact '{goal}' is directly known.")
        return True
    conditions = checkTarget(goal, rules)
    if not conditions:
        print(f"No rule or fact found to prove '{goal}'.")
        return False

    print(f"To prove '{goal}', checking required conditions: {conditions}")
    for cond in conditions:
        proven= backward_chaining(facts, rules, cond)
        if not proven:
            print(f"Failed to prove condition '{cond}'. Hypothesis '{goal}'
                rejected.")
    return False
```



```

print(f"All conditions met! Goal '{goal}' is PROVEN.")

return True


facts = [
    "Lights are dim",
    "Engine does not turn over"
]

rules = [
    (["Starter clicks", "Lights are dim"], "Dead Battery"),
    (["Engine turns over", "Fuel tank is empty"], "Out of Gas"),
    (["Engine does not turn over"], "Starter clicks")
]

target = "Dead Battery"

is_proven = backward_chaining(facts, rules, target)

print("=====")

print(f'Hypothesis '{target}': {'CONFIRMED' if is_proven else 'REJECTED'}\n")

```

OUTPUT

```

Granth Gupta@DESKTOP-OH4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-1/4_.py
To prove 'Dead Battery', checking required conditions: ['Starter clicks', 'Lights are dim']
To prove 'Starter clicks', checking required conditions: ['Engine does not turn over']
Fact 'Engine does not turn over' is directly known.
All conditions met! Goal 'Starter clicks' is PROVEN.
Fact 'Lights are dim' is directly known.
All conditions met! Goal 'Dead Battery' is PROVEN.
=====
Hypothesis 'Dead Battery': CONFIRMED

```

LAB-2 / QUESTION 1

Aim

Implement Forward and Backward Chaining for a given knowledge base and compare the reasoning mechanisms. (Expert Systems)

CODE

```
def checkFacts(conditions, facts):
```

```
    for cond in conditions:
```

```
        if cond not in facts:
```

```
            return False
```

```
    return True
```

```
# Forward Chain
```

```
def forward_chaining(facts, rules, goal):
```

```
    facts_changed = True
```

```
    while facts_changed:
```

```
        facts_changed = False
```

```
        for conditions, result in rules:
```

```
            if checkFacts(conditions, facts) and result not in facts:
```

```
                facts.append(result)
```

```
                facts_changed = True
```

```
        if result == goal:
```

```
            print(f'Goal '{goal}' successfully reached!')
```

```
            return True
```

```
    if goal in facts:
```

```
        return True
```

```
    else:
```

```
print(f'Could not reach goal '{goal}'.')  
return False
```

```
def checkTarget(target, rules):  
    for conditions, fact in rules:  
        if target == fact:  
            return conditions  
    return []
```

Backward chain

```
def backward_chaining(facts, rules, goal):  
    if goal in facts:  
        print(f'Fact '{goal}' is directly known.')  
        return True
```

```
conditions = checkTarget(goal, rules)
```

```
if not conditions:  
    print(f'No rule or fact found to prove '{goal}'.')  
  
    return False
```

```
print(f'To prove '{goal}', checking required conditions: {conditions}')
```

```
for cond in conditions:  
    proven= backward_chaining(facts, rules, cond)  
    if not proven:  
        print(f'Failed to prove condition '{cond}'. Hypothesis '{goal}'  
rejected.")  
    return False
```

```
    print(f'All conditions met! Goal '{goal}' is PROVEN.')
    return True
```

```
# Facts
```

```
facts = ["A", "B"]
```

```
# Rules:
```

```
rules = [
    (["A", "B"], "C"),
    (["C", "D"], "E"),
    (["C"], "F"),
    (["F"], "G")
]
```

```
target = "F"
```

```
# TESTING FORWARD CHAINING
```

```
print("=====")
print("FORWARD CHAINING TEST")
print("=====")
```

```
print("Test")
print("Initialise Facts: ", facts)
print("Taregt: ", target)
forward_chaining(facts, rules, target)
```

```
print("=====")
print("BACKWARD CHAINING TEST")
```

```
print("=====")
```

```
# TESTING BACKWARD CHAINING
```

```
is_proven = backward_chaining(facts, rules, target)
```

```
print("-----")
```

```
print(f'Hypothesis '{target}': {'CONFIRMED' if is_proven else 'REJECTED'}\n")
```

OUTPUT

```
Granth Gupta@DESKTOP-0H4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-2/1.py
=====
FORWARD CHAINING TEST
=====
Test
Initialise Facts: ['A', 'B']
Taregt: F
Goal 'F' successfully reached!
=====
BACKWARD CHAINING TEST
=====
Fact 'F' is directly known.
-----
Hypothesis 'F': CONFIRMED
```

LAB-2 / QUESTION 2

Aim

Implement BFS and DFS for solving a state-space problem and compare completeness, optimality, time and space requirements. (Blind Search)

CODE

```
from collections import deque
```

```
def add_edge(adj, u, v):
```

```
    adj[u].append(v)
```

```
    adj[v].append(u)
```

```
# 1. DEPTH-FIRST SEARCH (Recursive - Dives Deep First)
```

```
def dfs_rec(adj, visited, s, res):
```

```
    visited[s] = True
```

```
    res.append(s)
```

```
    for i in adj[s]:
```

```
        if not visited[i]:
```

```
            dfs_rec(adj, visited, i, res)
```

```
def dfs(adj):
```

```
    visited = [False] * len(adj)
```

```
    res = []
```

```
    dfs_rec(adj, visited, 0, res) # Start from node 0
```

```
    return res
```

```
# 2. BREADTH-FIRST SEARCH (Queue-based - Explores Level by Level)
```

```
def bfs(adj):
```

```
visited = [False] * len(adj)
```

```
res = []
```

```
q = deque()
```

```
src = 0
```

```
visited[src] = True
```

```
q.append(src)
```

```
while q:
```

```
    curr = q.popleft()
```

```
    res.append(curr)
```

```
    for x in adj[curr]:
```

```
        if not visited[x]:
```

```
            visited[x] = True
```

```
            q.append(x)
```

```
return res
```

```
if __name__ == "__main__":
```

```
    V = 5
```

```
    adj = [[] for _ in range(V)]
```

```
    # Adding edges
```

```
    add_edge(adj, 0, 1)
```

```
    add_edge(adj, 0, 2)
```

```
    add_edge(adj, 0, 4)
```

```
add_edge(adj, 2, 3)
```

```
# Perform DFS and BFS starting from node 0
```

```
dfs_res = dfs(adj)
```

```
bfs_res = bfs(adj)
```

```
print("DFS Traversal:", *dfs_res)
```

```
print("BFS Traversal:", *bfs_res)
```

OUTPUT

```
Granth Gupta@DESKTOP-OH4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-2/2.py
DFS Traversal: 0 1 2 3 4
BFS Traversal: 0 1 2 4 3
```


LAB-2 / QUESTION 3

Aim

Implement Greedy Best-First Search and A* for a path-finding problem and compare heuristic performance. (Heuristic Search)

CODE

```
import heapq
```

```
def manhattan_distance(node, goal):
```

```
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])
```

```
def get_neighbors(node, grid):
```

```
    neighbors = []
```

```
    rows, cols = len(grid), len(grid[0])
```

```
    r, c = node[0], node[1]
```

```
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    for dr, dc in moves:
```

```
        new_r, new_c = r + dr, c + dc
```

```
        # Check grid boundaries and make sure cell is not an obstacle (1)
```

```
        if 0 <= new_r < rows and 0 <= new_c < cols and grid[new_r][new_c] == 0:
```

```
            neighbors.append((new_r, new_c))
```

```
    return neighbors
```

```
def greedy_best_first_search(grid, start, goal):
```

```

# Priority Queue stores tuples: (heuristic_cost, current_node)

pq = []

heapq.heappush(pq, (manhattan_distance(start, goal), start))

visited = {start}

nodes_expanded = 0

print("--- Running Greedy Best-First Search ---")

while pq:
    # Always pop node with smallest heuristic h(n) value
    h_cost, current = heapq.heappop(pq)
    nodes_expanded += 1

    if current == goal:
        print(f"Goal Reached! Total Nodes Expanded: {nodes_expanded}")
        return True, nodes_expanded

    for neighbor in get_neighbors(current, grid):
        if neighbor not in visited:
            visited.add(neighbor)
            h = manhattan_distance(neighbor, goal)
            # Evaluation metric f(n) = h(n)
            heapq.heappush(pq, (h, neighbor))

print("Goal not reachable!")

return False, nodes_expanded

```

```

def a_star_search(grid, start, goal):
    # Priority Queue stores tuples: (f_cost, current_node)

    pq = []

    # g_score dictionary keeps track of shortest cost from start to each node
    g_score = {start: 0}

    initial_f = 0 + manhattan_distance(start, goal)
    heapq.heappush(pq, (initial_f, start))

    visited = set()
    nodes_expanded = 0

    print("--- Running A* Search ---")

    while pq:
        # Always pop node with smallest total cost  $f(n) = g(n) + h(n)$ 
        f_cost, current = heapq.heappop(pq)

        if current in visited:
            continue

        visited.add(current)
        nodes_expanded += 1

        if current == goal:

```

```
print(f'Goal Reached! Total Nodes Expanded: {nodes_expanded}')  
return True, nodes_expanded, g_score[current]
```

```
for neighbor in get_neighbors(current, grid):
```

```
    # Cost to move to neighbor cell is 1 unit step
```

```
    tentative_g = g_score[current] + 1
```

```
    # If neighbor reached via cheaper path or not visited before
```

```
    if neighbor not in g_score or tentative_g < g_score[neighbor]:
```

```
        g_score[neighbor] = tentative_g
```

```
        h = manhattan_distance(neighbor, goal)
```

```
        f = tentative_g + h #  $f(n) = g(n) + h(n)$ 
```

```
        heapq.heappush(pq, (f, neighbor))
```

```
print("Goal not reachable!")
```

```
return False, nodes_expanded, float("inf")
```

```
# 0 = Free Path, 1 = Obstacle
```

```
grid_map = [  
    [0, 0, 0, 0, 0],  
    [0, 1, 1, 1, 0],  
    [0, 0, 0, 1, 0],  
    [1, 1, 0, 0, 0],  
    [0, 0, 0, 0, 0]  
]
```

```
start_pos = (0, 0)
```

```
goal_pos = (4, 4)
```

```
greedy_found, greedy_nodes = greedy_best_first_search(grid_map, start_pos, goal_pos)
```

```
print("\n" + "="*45 + "\n")
```

```
a_star_found, a_star_nodes, path_cost = a_star_search(grid_map, start_pos, goal_pos)
```

```
print(f"Shortest Path Cost: {path_cost}")
```

OUTPUT

```
Granth Gupta@DESKTOP-OH4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-2/3.py
--- Running Greedy Best-First Search ---
Goal Reached! Total Nodes Expanded: 9

=====

--- Running A* Search ---
Goal Reached! Total Nodes Expanded: 17
Shortest Path Cost: 8
```

LAB-2 / QUESTION 4

Aim

Solve the 8-Puzzle Problem using A* and analyze nodes generated, nodes expanded, solution depth and solution cost.(state-space)

CODE

```
import heapq

def manhattan_distance(board):
    distance = 0

    # Map each tile to its target (row, col) in goal_board:
    # 1=(0,0), 2=(0,1), 3=(0,2), 4=(1,0), 5=(1,1), 6=(1,2), 7=(2,0), 8=(2,1)
    goal_positions = {
        1: (0, 0), 2: (0, 1), 3: (0, 2),
        4: (1, 0), 5: (1, 1), 6: (1, 2),
        7: (2, 0), 8: (2, 1)
    }

    for i in range(9):
        tile = board[i]
        if tile != 0:
            current_row, current_col = divmod(i, 3)
            goal_row, goal_col = goal_positions[tile]
            distance += abs(current_row - goal_row) + abs(current_col - goal_col)

    return distance

def get_neighbors(board):
    neighbors = []

    zero_idx = board.index(0) # Index of blank space
```

```
row, col = divmod(zero_idx, 3)
```

```
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # UP, DOWN, LEFT, RIGHT
```

```
for dr, dc in moves:
```

```
    r, c = row + dr, col + dc
```

```
    if 0 <= r < 3 and 0 <= c < 3:
```

```
        new_zero = r * 3 + c
```

```
        new_board = list(board)
```

```
        # Swap tile with blank space
```

```
        new_board[zero_idx], new_board[new_zero] = new_board[new_zero], new_board[zero_idx]
```

```
        neighbors.append(tuple(new_board))
```

```
return neighbors
```

```
def solve_a_star(start_board, goal_board):
```

```
    # Priority Queue stores tuples: (f_score, counter, current_board, g_score)
```

```
    # Counter breaks ties in priority queue
```

```
    counter = 0
```

```
    pq = []
```

```
    h_start = manhattan_distance(start_board)
```

```
    heapq.heappush(pq, (h_start, counter, start_board, 0))
```

```
    g_score = {start_board: 0}
```

```
    visited = set()
```

```

# Performance Counters

nodes_expanded = 0

nodes_generated = 1 # Includes start node


while pq:

    f, _, current, g = heapq.heappop(pq)

    if current in visited:

        continue

    visited.add(current)

    nodes_expanded += 1


    if current == goal_board:

        solution_depth = g

        solution_cost = g # Step cost per tile slide = 1

        return nodes_generated, nodes_expanded, solution_depth, solution_cost


    for neighbor in get_neighbors(current):

        tentative_g = g + 1

        if neighbor not in g_score or tentative_g < g_score[neighbor]:

            g_score[neighbor] = tentative_g

            h = manhattan_distance(neighbor)

            f_score = tentative_g + h

            counter += 1

            nodes_generated += 1

            heapq.heappush(pq, (f_score, counter, neighbor, tentative_g))

```



```

    return nodes_generated, nodes_expanded, -1, -1

if __name__ == "__main__":
    initial_state = (
        1, 2, 3,
        4, 0, 6,
        7, 5, 8
    )
    goal_state = (
        1, 2, 3,
        4, 5, 6,
        7, 8, 0
    )
    gen, exp, depth, cost = solve_a_star(initial_state, goal_state)
    print("--- A* Search Analysis Results ---")
    print(f"Nodes Generated : {gen}")
    print(f"Nodes Expanded : {exp}")
    print(f"Solution Depth : {depth}")
    print(f"Solution Cost : {cost}")

```

OUTPUT

```

Granth Gupta@DESKTOP-0H4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-2/4.py
--- A* Search Analysis Results ---
Nodes Generated : 7
Nodes Expanded : 3
Solution Depth : 2
Solution Cost : 2

```

LAB-2 / QUESTION 5

Aim

Write a program that prints $g(n)$, $h(n)$ and $f(n) = g(n) + h(n)$ at every city along the two paths given below, and also prints the actual total cost of each path. Check whether $h(n)$ ever becomes larger than the cost actually remaining on that path. Path 1 : A -> C -> E -> F -> G Path 2 : A -> C -> B -> D -> E -> F -> G (Heuristic Evaluation Function)

CODE

```
edge_costs = {  
    ('A', 'C'): 2,  
    ('C', 'E'): 4,  
    ('E', 'F'): 3,  
    ('F', 'G'): 2,  
    ('C', 'B'): 1,  
    ('B', 'D'): 2,  
    ('D', 'E'): 3  
}
```

Heuristic estimates $h(n)$ to Goal city G

```
heuristic = {  
    'A': 8,  
    'B': 6,  
    'C': 7,  
    'D': 4,  
    'E': 3,  
    'F': 1,  
    'G': 0  
}
```

```

def evaluate_path(path_name, path):

    print(f"EVALUATING: {path_name} ( Path: {' -> '.join(path)} )")

    # Step 1: Calculate cumulative path cost g(n) for each node

    g_values = {}

    g_values[path[0]] = 0 # Start node A has g(A) = 0

    for i in range(1, len(path)):

        prev_node = path[i-1]

        curr_node = path[i]

        edge = (prev_node, curr_node)

        # Add edge cost to previous g(n) value

        g_values[curr_node] = g_values[prev_node] + edge_costs[edge]

    total_path_cost = g_values[path[-1]]

    # Header for tabular output

    print(f"{'City (n)':<10} | {'g(n)':<8} | {'h(n)':<8} | {'f(n)=g+h':<10} | {'Actual Remaining':<18} |  

{'Is h(n) > Remaining?'}")

    print("-" * 80)

    admissible_violation = False

    # Step 2: Evaluate metrics at each city along the path

```

```

for city in path:
    g = g_values[city]
    h = heuristic[city]
    f = g + h

    # Actual cost remaining from current city to Goal G
    actual_remaining = total_path_cost - g

    # Check if heuristic overestimates actual remaining cost
    overestimated = h > actual_remaining
    if overestimated:
        admissible_violation = True

    overestimate_str = "YES (Inadmissible)" if overestimated else "No (Admissible)"

    print(f"{city:<10} | {g:<8} | {h:<8} | {f:<10} | {actual_remaining:<18} | {overestimate_str}")

print("-" * 80)
print(f"TOTAL PATH COST: {total_path_cost}")

if admissible_violation:
    print("WARNING: h(n) became larger than actual remaining cost at one or more nodes!")
else:
    print("SUCCESS: h(n) NEVER exceeded actual remaining cost on this path.")

return total_path_cost

```

```

if __name__ == "__main__":

    # Define paths

    path1 = ['A', 'C', 'E', 'F', 'G']

    path2 = ['A', 'C', 'B', 'D', 'E', 'F', 'G']


    # Run evaluations

    cost1 = evaluate_path("Path 1", path1)

    cost2 = evaluate_path("Path 2", path2)

```

OUTPUT

```

Granth Gupta@DESKTOP-0H4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-2/5.py
EVALUATING: Path 1 ( Path: A -> C -> E -> F -> G )
City (n) | g(n) | h(n) | f(n)=g+h | Actual Remaining | Is h(n) > Remaining?
-----
A | 0 | 8 | 8 | 11 | No (Admissible)
C | 2 | 7 | 9 | 9 | No (Admissible)
E | 6 | 3 | 9 | 5 | No (Admissible)
F | 9 | 1 | 10 | 2 | No (Admissible)
G | 11 | 0 | 11 | 0 | No (Admissible)
-----
TOTAL PATH COST: 11
SUCCESS: h(n) NEVER exceeded actual remaining cost on this path.
EVALUATING: Path 2 ( Path: A -> C -> B -> D -> E -> F -> G )
City (n) | g(n) | h(n) | f(n)=g+h | Actual Remaining | Is h(n) > Remaining?
-----
A | 0 | 8 | 8 | 13 | No (Admissible)
C | 2 | 7 | 9 | 11 | No (Admissible)
B | 3 | 6 | 9 | 10 | No (Admissible)
D | 5 | 4 | 9 | 8 | No (Admissible)
E | 8 | 3 | 11 | 5 | No (Admissible)
F | 11 | 1 | 12 | 2 | No (Admissible)
G | 13 | 0 | 13 | 0 | No (Admissible)
-----
TOTAL PATH COST: 13
SUCCESS: h(n) NEVER exceeded actual remaining cost on this path.

```

LAB-2 / QUESTION 6

Aim

To implement Greedy Best First Search on the given road map using a priority queue ordered by $h(n)$ alone. Find a path from A to G, print the cities in the order they are expanded along with their h values, then print the path found and its total cost.

Code

```
import heapq

print("GREEDY BEST FIRST SEARCH")
print("-----")

# Road map
graph = {
    "A": {"B": 4, "C": 2},
    "B": {"A": 4, "C": 1, "D": 5},
    "C": {"A": 2, "B": 1, "D": 8, "E": 10},
    "D": {"C": 8, "B": 5, "F": 6, "E": 2},
    "E": {"C": 10, "D": 2, "F": 3},
    "F": {"D": 6, "E": 3, "G": 1},
    "G": {"F": 1}
}

# Heuristic values
h = {
    "A": 13,
    "B": 11,
    "C": 9,
    "D": 5,
    "E": 3,
    "F": 1,
    "G": 0
}

start = "A"
goal = "G"

# Priority queue
queue = [(h[start], start, [start], 0)]
```

```
visited = set()
```

```
while queue:
```

```
    heuristic, city, path, cost = heapq.heappop(queue)
```

```
    if city in visited:
```

```
        continue
```

```
    visited.add(city)
```

```
    print("Expanded:", city, "h =", h[city])
```

```
    # Goal reached
```

```
    if city == goal:
```

```
        print("\nPath:", " -> ".join(path))
```

```
        print("Total cost:", cost)
```

```
        break
```

```
    # Add neighboring cities
```

```
    for next_city in graph[city]:
```

```
        if next_city not in visited:
```

```
            new_cost = cost + graph[city][next_city]
```

```
            new_path = path + [next_city]
```

```
            heapq.heappush(
```

```
                queue,
```

```
                (h[next_city], next_city, new_path, new_cost)
```

```
            )
```

OUTPUT

```
Granth Gupta@DESKTOP-OH4NI9V MINGW64 /d/NITJ Study/AI Lab (main)
$ python Lab-2/6.py
GREEDY BEST FIRST SEARCH
-----
Expanded: A h = 13
Expanded: C h = 9
Expanded: E h = 3
Expanded: F h = 1
Expanded: G h = 0

Path: A -> C -> E -> F -> G
Total cost: 16
```